МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ОТЧЁТ

по лабораторной работе № 6

«Реализация линейных структур данных и анализ трудоёмкости их операций»

Дисциплина: **Теория алгоритмов**

Направление подготовки: 38.03.05 «Бизнес-информатика»

Вариант 13

Выполнила: студентка группы БИ 2-1

Кугушева Арина Вячеславовна

Проверил: _____

2026

1. Цель работы и задание варианта

Цель работы: изучить линейные структуры данных (связный список, очередь на кольцевом буфере), реализовать их и экспериментально установить, как выбор структуры определяет трудоёмкость типовых операций.

Задание (вариант 13). Реализовать **двусвязный список** и **кольцевую очередь**. Измерить время **вставки в середину** и **обхода в обе стороны** для собственной реализации и для встроенного `list` при n = 1000, 2000, 4000. Решить прикладную задачу «**плей-лист с переходами**».

№	Структуры	Исследуемые операции	Размеры n	Прикладная задача
13	Двусвязный список, кольцевая очередь	Вставка в середину, обход в обе стороны	1000; 2000; 4000	Плей-лист с переходами

Дополнительные требования (для всех вариантов): T1 — проверка на малом примере с выводом содержимого после каждой операции; T2 — измерение времени каждой операции для своей реализации и для `list`; T3 — графики T(n) и определение соответствия оценкам Θ(1)/Θ(n); T4 — оценка расхода памяти на 10 000 элементов с помощью `sys.getsizeof`.

Условия эксперимента. Время измерялось функцией `time.perf_counter()`; результат — медиана по 7 повторам после одного прогревочного запуска, на время замера сборщик мусора отключался. Подготовка данных (построение структуры) во время замера не входит. Вставка в середину: время одной операции по серии из 100 вставок в позицию len/2 (для list — по серии из 1000; для вставки после известного узла — из 1000). Обход: суммарное время прохода по всем n элементам. Доступ по индексу: среднее время одного из 200 обращений к случайным позициям. Кроме времени подсчитывается число **переходов по ссылкам** («шагов») — машинно-независимая мера, которую можно сопоставлять с теорией напрямую.

Дополнительно (помимо варианта) для наглядности исследованы: вставка после уже известного узла, обход назад «без ссылок prev» (через доступ по индексу от головы), доступ по индексу, очередь в сравнении с `collections.deque` и `list.pop(0)`, расход памяти (T4).

2. Описание реализованных структур и их операций

Двусвязный список (класс `DoublyLinkedList`). Каждый узел `Node` хранит значение и две ссылки — `prev` и `next`; список хранит ссылки на голову `head` и хвост `tail`. Реализованы операции: `push_front`, `push_back`, `insert(i, v)`, `insert_after(node, v)`, `pop_front`, `pop_back`, `delete(i)`, `remove_node(node)`, `get(i)`, `find(v)`, обход вперёд (`for x in list`) и назад (`reversed()`).

Вставка и удаление сводятся к переустановке ссылок и стоят Θ(1), **если узел известен**. Доступ к i-му элементу требует прохода по ссылкам: благодаря двум ссылкам проход начинается с ближайшего конца, поэтому число переходов равно min(i, n − 1 − i), то есть до n/2 (в среднем n/4 при случайном i). Вставка в *середину по индексу* — Θ(n): ≈ n/2 переходов до позиции плюс Θ(1) на перестановку четырёх ссылок. Наличие ссылки `prev` даёт удаление из конца за Θ(1) и обход назад за Θ(n); в односвязном списке обход назад потребовал бы $\Theta(n^2)$ переходов (доступ к каждому элементу с головы: 0 + 1 + … + (n − 1) = n(n − 1)/2 — это измерено в эксперименте).

Кольцевая очередь (класс `CircularQueue`) — FIFO на массиве фиксированной ёмкости m. Хранятся индекс головы `head` и число элементов `count`; позиция записи tail = (head + count) mod m, поэтому после последней ячейки запись продолжается с

нулевой. `enqueue` и `dequeue` выполняются за Θ(1) без сдвига элементов; при переполнении и извлечении из пустой очереди выбрасываются исключения. В обычном массиве извлечение из начала (`list.pop(0)`) требует Θ(n) на сдвиг.

Плей-лист (класс `Playlist`): треки лежат в двусвязном списке, текущий трек — ссылка `current` на узел. Реализованы переходы `next_track`/`prev_track` (с зацикливанием), вставка трека после текущего, удаление текущего, очередь «играть следующим» на кольцевом буфере (`queue_next`, `play_next`).

Таблица 1. Теоретическая трудоёмкость операций

Операция	Двусвязный список	Кольцевая очередь	Массив (list)
Доступ по индексу	Θ(n) (до n/2 переходов)	—	Θ(1)
Вставка в начало / конец	Θ(1)	enqueue: Θ(1)	Θ(n) / O(1) аморт.
Вставка в середину по индексу	Θ(n) (поиск позиции)	—	Θ(n) (сдвиг)
Вставка после известного узла	Θ(1)	—	—
Удаление из начала / конца	Θ(1) / Θ(1)	dequeue: Θ(1)	Θ(n) / Θ(1)
Обход вперёд / назад	Θ(n) / Θ(n)	—	Θ(n) / Θ(n)
Поиск значения	Θ(n)	Θ(n)	Θ(n)
Память на элемент	данные + 2 ссылки + заголовок узла	данные + ячейка	данные + ссылка

3. Схемы структур данных и блок-схемы алгоритмов

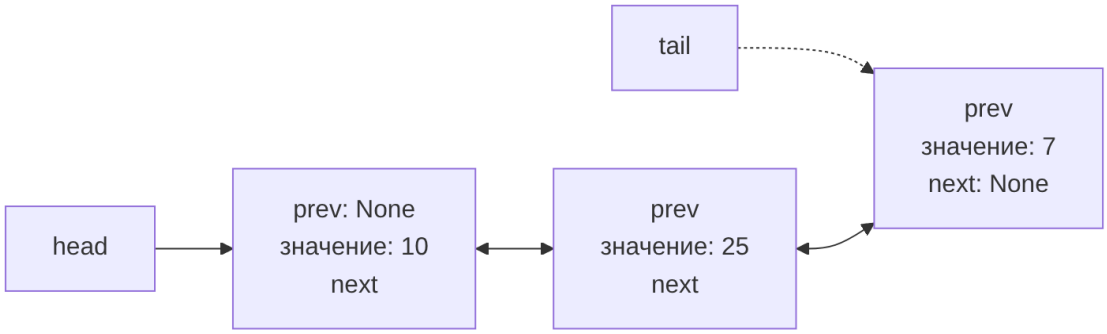


Рис. 1. Схема двусвязного списка (узлы и ссылки prev / next)

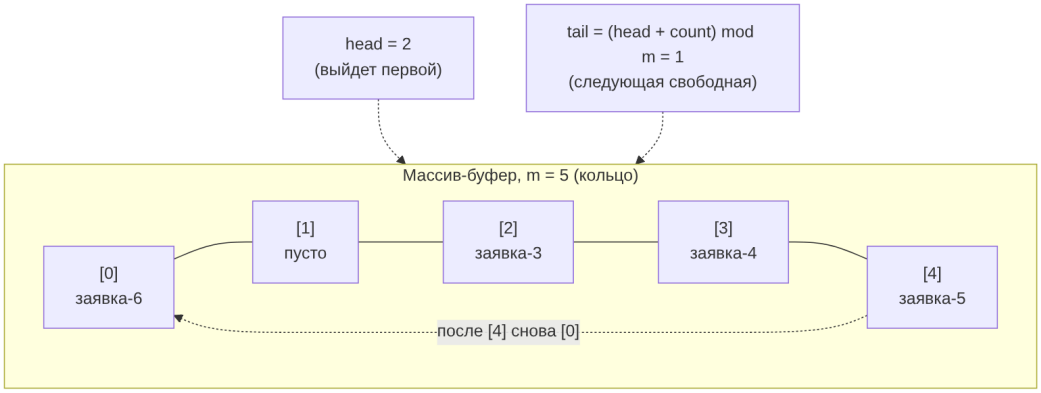


Рис. 2. Схема кольцевого буфера (m = 5): tail вычисляется по модулю m, поэтому запись переходит через конец массива

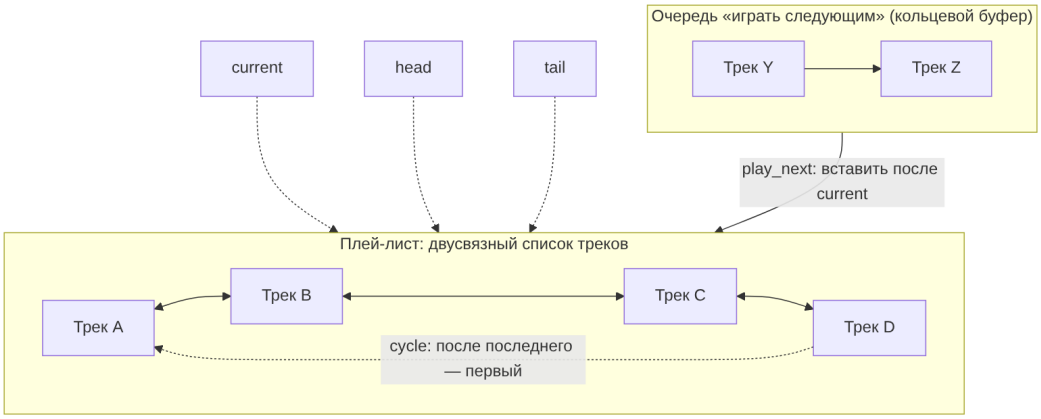


Рис. 3. Схема плей-листа: двусвязный список треков, указатель current, зацикливание и очередь «играть следующим»

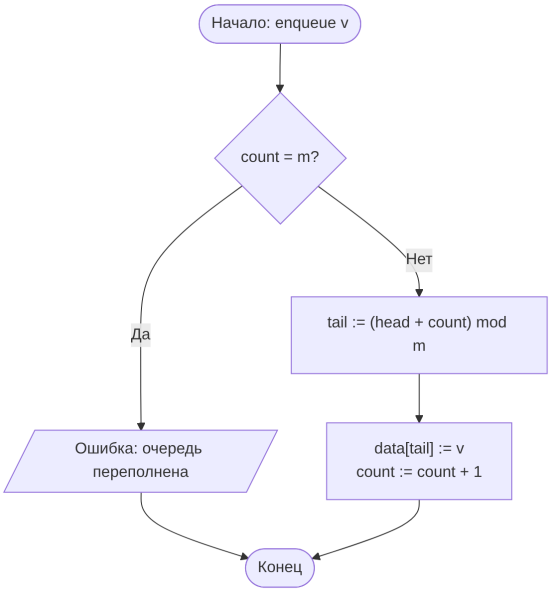


Рис. 4. Блок-схема постановки в кольцевую очередь (enqueue)

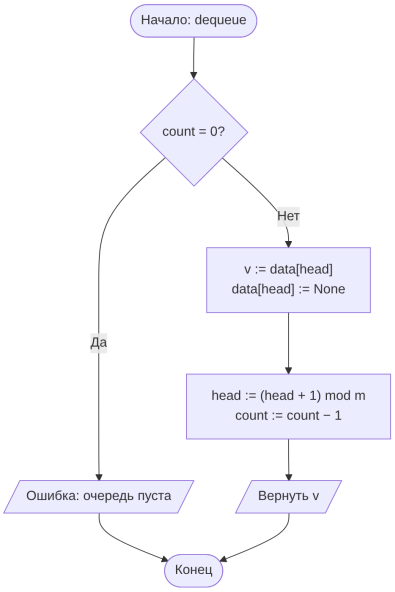


Рис. 5. Блок-схема извлечения из кольцевой очереди (dequeue)

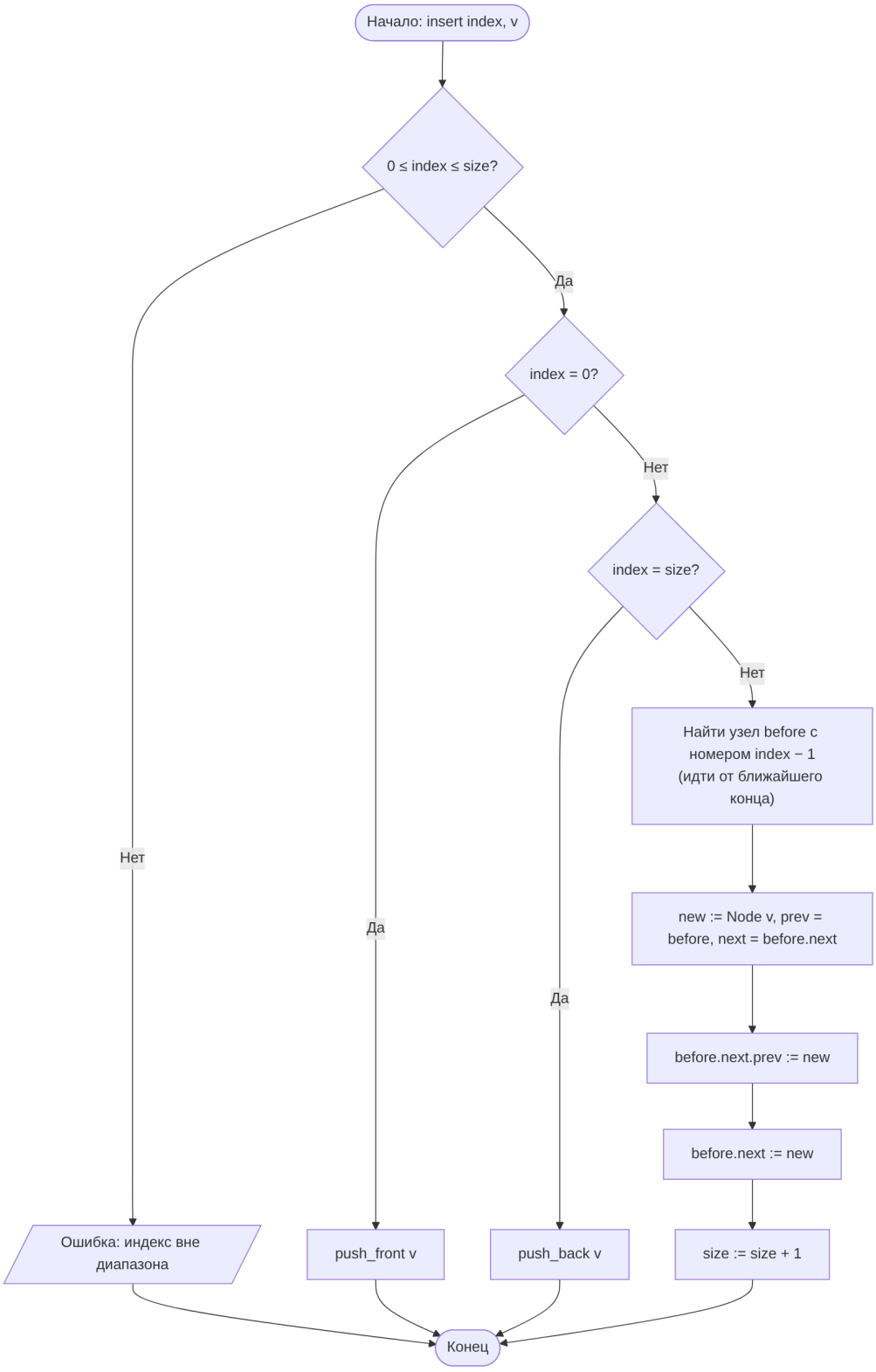


Рис. 6. Блок-схема вставки в двусвязный список по индексу (insert)

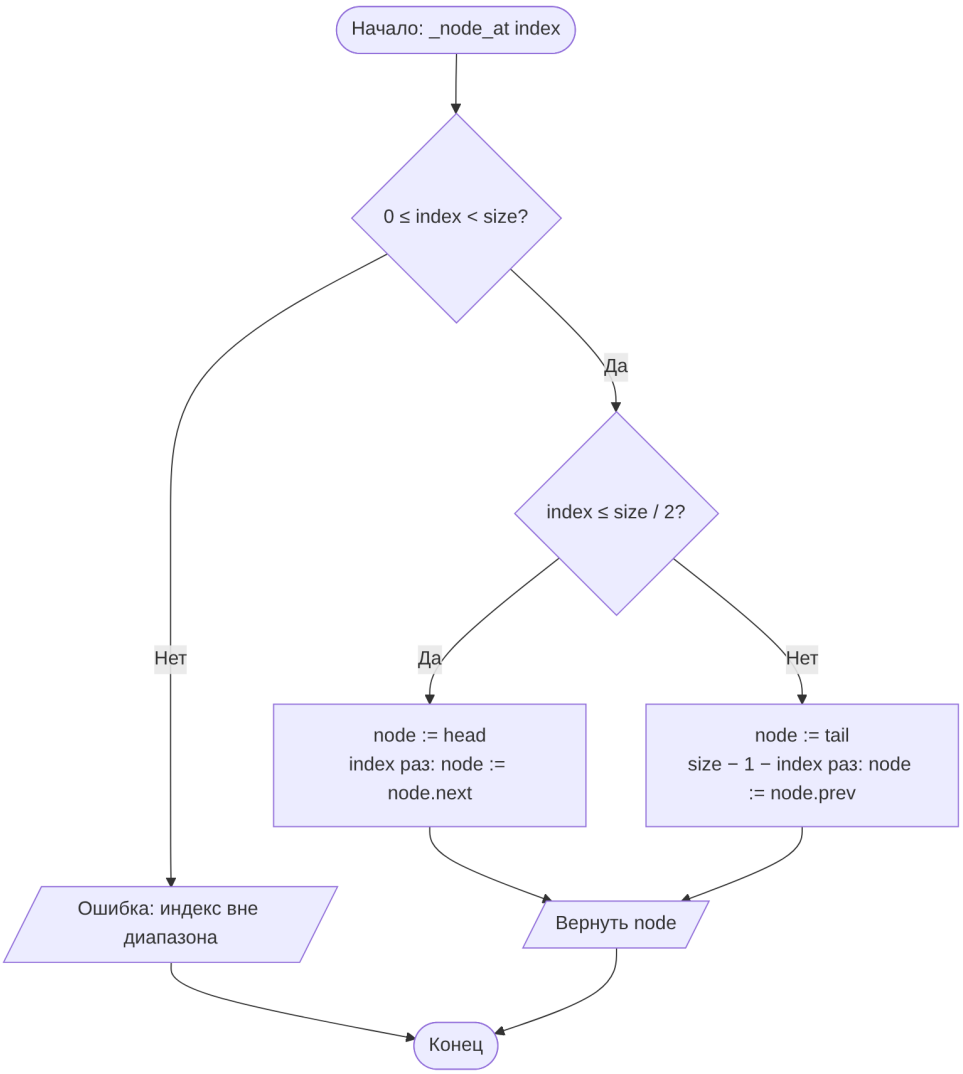


Рис. 7. Блок-схема поиска узла по индексу с ближайшего конца (_node_at)

4. Листинг программы

Файл `lr6_variant13.py` (самодостаточный модуль с подробными комментариями: структуры, плей-лист, самопроверка, малый пример, измерения, память, графики). Запуск: `python lr6_variant13.py`; требуется matplotlib.

```
"""
Лабораторная работа № 6. Вариант 13.
Дисциплина: Теория алгоритмов.
Выполнила: Кугушева Арина Вячеславовна, группа БИ 2-1.

Тема: реализация линейных структур данных и анализ трудоёмкости их операций.

ИСХОДНЫЕ ДАННЫЕ ВАРИАНТА 13 (табл. 1 задания)
    структуры для реализации ... двусвязный список и кольцевая очередь
                                 (очередь на основе кольцевого буфера);
    исследуемые операции ....... вставка в середину, обход в обе стороны;
    размеры n .................. 1000, 2000, 4000;
    прикладная задача .......... плей-лист с переходами (вперёд / назад).

ДОПОЛНИТЕЛЬНЫЕ ТРЕБОВАНИЯ (табл. 2, для всех вариантов)
    T1  проверка структур на малом примере с выводом содержимого после
        каждой операции;
    T2  измерение времени каждой операции для собственной реализации и для
        встроенного list;
    T3  графики T(n) и определение, какой оценке (θ(1), θ(n), θ(n²))
        соответствует каждая кривая;
    T4  оценка расхода памяти на 10 000 элементов (sys.getsizeof).

ЧТО ДЕЛАЕТ ПРОГРАММА (по порядку)
    0. Самопроверка: списки и очередь сравниваются со встроенными
       list и collections.deque на случайных операциях.
    1. T1 - малый пример: содержимое структур печатается после каждой операции.
    2. Прикладная задача: плей-лист на двусвязном списке с кольцевой
       очередью «играть следующим» (сценарий с выводом).
    3. Измерения (T2): вставка в середину, обход вперёд и назад, доступ по
       индексу, кольцевая очередь; кроме времени считается число переходов
       по ссылкам («шагов»), которое не зависит от скорости компьютера.
    4. T3: отношения T(2n)/T(n) и показатель наклона в двойных логарифмических
       координатах, графики lr6_plot.png.
    5. T4: расход памяти на 10 000 элементов, график lr6_plot_memory.png.

ЗАПУСК
    python lr6_variant13.py             (полный прогон, около 1 минуты)
    python lr6_variant13.py --json F    (дополнительно сохранить результаты в F)
    python lr6_variant13.py --show      (дополнительно показать графики)

Требуется: Python 3.10+, библиотека matplotlib (pip install matplotlib).
"""

import gc
import json
import math
import platform
import random
import statistics
import sys
import time
import tracemalloc
from collections import deque

import matplotlib

if "--show" not in sys.argv:
    matplotlib.use("Agg")          # режим без окна: графики только сохраняются в файлы
import matplotlib.pyplot as plt

# ---------------------------------------------------------------------------
# 1. ПАРАМЕТРЫ ЭКСПЕРИМЕНТА
# ---------------------------------------------------------------------------
SIZES = [1000, 2000, 4000]          # размеры n из варианта 13
EXT_LIST_SIZES = [10**4, 10**5, 10**6]   # дополнительные размеры для вставки в середину list
EXT_QUEUE_SIZES = [16_000, 64_000]       # дополнительные размеры для сравнения очередей
K_MID = 100                         # число вставок в середину на один замер
K_NODE = 1000                       # число вставок после известного узла на один замер
K_ACCESS = 200                      # число обращений по индексу на один замер
REPS = 7                            # число повторов замера (берётся медиана)
SEED = 13                           # начальное значение генератора случайных чисел
MEM_N = 10_000                      # число элементов для оценки памяти (T4)


# ---------------------------------------------------------------------------
```

```
# 2. ДВУСВЯЗНЫЙ СПИСОК
# ------------------------------------------------------------------------
class Node:
    """
    Узел двусвязного списка: значение и две ссылки - на предыдущий (prev)
    и следующий (next) узлы. __slots__ запрещает создание словаря атрибутов
    у каждого узла, что заметно экономит память.
    """
    __slots__ = ("value", "prev", "next")

    def __init__(self, value, prev=None, nxt=None):
        self.value = value          # хранимые данные
        self.prev = prev            # ссылка на предыдущий узел (None у головы)
        self.next = nxt             # ссылка на следующий узел (None у хвоста)


class DoublyLinkedList:
    """
    Двусвязный список со ссылками на голову (head) и хвост (tail).

    Счётчик self.hops считает число переходов по ссылкам при поиске узла по
    индексу и при «ручном» доступе с головы. Это машинно-независимая мера
    трудоёмкости, которую можно сопоставлять с теорией.

    Трудоёмкость основных операций:
        push_front, push_back, pop_front, pop_back ....... θ(1)
        insert_after(node), remove_node(node) ............ θ(1) - узел известен
        get(i), insert(i, v), delete(i) .................. θ(min(i, n - i))
        find(v) ........................................... θ(n)
        обход вперёд / назад ............................. θ(n)
    """

    def __init__(self):
        self.head = None            # первый узел
        self.tail = None            # последний узел
        self.size = 0               # число элементов
        self.hops = 0               # счётчик переходов по ссылкам

    def __len__(self):
        return self.size

    # ---------------------------- обход -----------------------------------
    def __iter__(self):
        """Обход ВПЕРЁД от головы к хвосту по ссылкам next: θ(n)."""
        node = self.head
        while node is not None:
            yield node.value
            node = node.next

    def reversed(self):
        """Обход НАЗАД от хвоста к голове по ссылкам prev: θ(n)."""
        node = self.tail
        while node is not None:
            yield node.value
            node = node.prev

    def to_list(self):
        """Содержимое списка (слева направо) в виде обычного list."""
        return list(self)

    def to_list_reversed(self):
        """Содержимое списка (справа налево) в виде обычного list."""
        return list(self.reversed())

    # ------------------------ доступ по индексу ----------------------------
    def _node_at(self, index):
        """
        Возвращает узел с номером index. Благодаря двум ссылкам идём с ТОГО
        конца, который ближе: максимум n/2 переходов вместо n.
        """
        if not 0 <= index < self.size:
            raise IndexError("Индекс вне диапазона")
        if index <= self.size // 2:                 # ближе к голове
            node = self.head
            for _ in range(index):
                node = node.next
            self.hops += index
        else:                                       # ближе к хвосту
            node = self.tail
            steps = self.size - 1 - index
            for _ in range(steps):
                node = node.prev
            self.hops += steps
        return node
    def get_from_head(self, index):
```

```
    """
    Доступ по индексу ТОЛЬКО от головы (так работал бы односвязный список,
    у которого нет ссылки prev). Нужен для сравнения: обход назад через
    такие обращения стоит θ(n²).
    """
    if not 0 <= index < self.size:
        raise IndexError("Индекс вне диапазона")
    node = self.head
    for _ in range(index):
        node = node.next
    self.hops += index
    return node.value

def get(self, index):
    """Значение по индексу: θ(min(i, n - i))."""
    return self._node_at(index).value

def find(self, value):
    """Индекс первого вхождения value или -1: θ(n)."""
    node, i = self.head, 0
    while node is not None:
        if node.value == value:
            return i
        node, i = node.next, i + 1
    return -1

# ------------------------------ вставка ---------------------------------
def push_front(self, value):
    """Вставка в начало: θ(1)."""
    node = Node(value, None, self.head)         # новый узел перед головой
    if self.head is not None:
        self.head.prev = node
    else:                                       # список был пуст
        self.tail = node
    self.head = node
    self.size += 1

def push_back(self, value):
    """Вставка в конец: θ(1) благодаря ссылке tail."""
    node = Node(value, self.tail, None)         # новый узел после хвоста
    if self.tail is not None:
        self.tail.next = node
    else:                                       # список был пуст
        self.head = node
    self.tail = node
    self.size += 1

def insert_after(self, node, value):
    """
    Вставка нового значения СРАЗУ ПОСЛЕ известного узла: θ(1).
    Переставляются только четыре ссылки, сдвигать элементы не нужно.
    """
    new = Node(value, node, node.next)          # prev = node, next = node.next
    if node.next is not None:
        node.next.prev = new                    # следующий узел указывает назад на new
    else:                                       # node был хвостом
        self.tail = new
    node.next = new
    self.size += 1
    return new

def insert(self, index, value):
    """
    Вставка значения так, чтобы оно получило номер index (0 <= index <= n).
    Для середины списка нужно дойти до позиции: θ(n / 2) переходов,
    сама перестановка ссылок - θ(1).
    """
    if not 0 <= index <= self.size:
        raise IndexError("Индекс вне диапазона")
    if index == 0:
        self.push_front(value)
    elif index == self.size:
        self.push_back(value)
    else:
        before = self._node_at(index - 1)       # узел, после которого вставляем
        self.insert_after(before, value)

# ------------------------------ удаление --------------------------------
def remove_node(self, node):
    """Удаление известного узла: θ(1). Возвращает его значение."""
    if node.prev is not None:
        node.prev.next = node.next              # предыдущий «перепрыгивает» узел
    else:                                       # удаляем голову
        self.head = node.next
    if node.next is not None:
```

```
            node.next.prev = node.prev              # следующий ссылается на предыдущий
        else:                                       # удаляем хвост
            self.tail = node.prev
        self.size -= 1
        return node.value

    def pop_front(self):
        """Удаление из начала: θ(1)."""
        if self.head is None:
            raise IndexError("Список пуст")
        return self.remove_node(self.head)

    def pop_back(self):
        """Удаление из конца: θ(1) (в односвязном списке было бы θ(n))."""
        if self.tail is None:
            raise IndexError("Список пуст")
        return self.remove_node(self.tail)

    def delete(self, index):
        """Удаление по индексу: θ(min(i, n - i)). Возвращает значение."""
        return self.remove_node(self._node_at(index))


# ---------------------------------------------------------------------------
# 3. КОЛЬЦЕВАЯ ОЧЕРЕДЬ
# ---------------------------------------------------------------------------
class CircularQueue:
    """
    Очередь (FIFO) на основе КОЛЬЦЕВОГО БУФЕРА фиксированной ёмкости.

    Хранится массив из capacity ячеек, индекс головы (head) и число элементов
    (count). Индекс следующей свободной ячейки (tail) вычисляется как
        tail = (head + count) mod capacity,
    поэтому при достижении конца массива запись продолжается с его начала.
    enqueue и dequeue работают за θ(1) и не сдвигают элементы.
    """

    def __init__(self, capacity):
        if capacity <= 0:
            raise ValueError("Ёмкость должна быть положительной")
        self.data = [None] * capacity   # массив-буфер
        self.capacity = capacity        # ёмкость буфера
        self.head = 0                   # индекс элемента, который выйдет первым
        self.count = 0                  # число элементов в очереди

    def __len__(self):
        return self.count

    def is_empty(self):
        return self.count == 0

    def is_full(self):
        return self.count == self.capacity

    def enqueue(self, value):
        """Постановка в очередь: θ(1). При переполнении - OverflowError."""
        if self.is_full():
            raise OverflowError("Очередь переполнена")
        tail = (self.head + self.count) % self.capacity     # следующая свободная ячейка
        self.data[tail] = value
        self.count += 1

    def dequeue(self):
        """Извлечение из очереди: θ(1). Из пустой очереди - IndexError."""
        if self.is_empty():
            raise IndexError("Очередь пуста")
        value = self.data[self.head]
        self.data[self.head] = None                         # освобождаем ссылку
        self.head = (self.head + 1) % self.capacity         # голова двигается по кругу
        self.count -= 1
        return value

    def front(self):
        """Первый элемент без извлечения: θ(1)."""
        if self.is_empty():
            raise IndexError("Очередь пуста")
        return self.data[self.head]

    def to_list(self):
        """Содержимое очереди от головы к хвосту (для вывода)."""
        return [self.data[(self.head + i) % self.capacity] for i in range(self.count)]


# ---------------------------------------------------------------------------
# 4. ПРИКЛАДНАЯ ЗАДАЧА: ПЛЕЙ-ЛИСТ С ПЕРЕХОДАМИ
```

```
# ----------------------------------------------------------------------------
class Playlist:
    """
    Плей-лист: треки хранятся в двусвязном списке, текущий трек - ссылка
    current на узел списка.

    Переходы «следующий» и «предыдущий», вставка трека сразу после текущего
    и удаление текущего выполняются за θ(1): позиция уже известна (это узел),
    искать по индексу не нужно. В режиме cycle=True после последнего трека
    воспроизведение продолжается с первого (и наоборот при переходе назад).

    Очередь «играть следующим» - кольцевая очередь (CircularQueue): трек,
    поставленный в неё, будет вставлен после текущего при ближайшем
    переходе вперёд, раньше остальных треков плей-листа.
    """

    def __init__(self, titles=(), cycle=True, queue_capacity=8):
        self.tracks = DoublyLinkedList()            # список треков
        for title in titles:
            self.tracks.push_back(title)
        self.current = self.tracks.head             # текущий трек (узел)
        self.cycle = cycle                          # зацикливать воспроизведение
        self.up_next = CircularQueue(queue_capacity)    # очередь «играть следующим»

    def now_playing(self):
        """Название текущего трека (или None, если плей-лист пуст)."""
        return None if self.current is None else self.current.value

    def next_track(self):
        """Переход к следующему треку по ссылке next: θ(1)."""
        if self.current is None:
            return None
        nxt = self.current.next
        if nxt is None and self.cycle:              # дошли до конца - переходим к началу
            nxt = self.tracks.head
        if nxt is not None:
            self.current = nxt
        return self.current.value

    def prev_track(self):
        """Переход к предыдущему треку по ссылке prev: θ(1)."""
        if self.current is None:
            return None
        prv = self.current.prev
        if prv is None and self.cycle:              # дошли до начала - переходим к концу
            prv = self.tracks.tail
        if prv is not None:
            self.current = prv
        return self.current.value

    def add_to_end(self, title):
        """Добавить трек в конец плей-листа: θ(1)."""
        self.tracks.push_back(title)
        if self.current is None:
            self.current = self.tracks.head

    def add_after_current(self, title):
        """Вставить трек сразу после текущего: θ(1) (в массиве потребовался бы сдвиг)."""
        if self.current is None:
            self.add_to_end(title)
        else:
            self.tracks.insert_after(self.current, title)

    def remove_current(self):
        """Удалить текущий трек и перейти к следующему (или предыдущему): θ(1)."""
        if self.current is None:
            return None
        node = self.current
        self.current = node.next if node.next is not None else node.prev
        return self.tracks.remove_node(node)

    def queue_next(self, title):
        """Поставить трек в очередь «играть следующим»."""
        self.up_next.enqueue(title)

    def play_next(self):
        """
        Переход вперёд с учётом очереди: если в очереди «играть следующим» есть
        треки, первый из них вставляется после текущего и становится текущим;
        иначе выполняется обычный переход к следующему треку.
        """
        if not self.up_next.is_empty():
            self.add_after_current(self.up_next.dequeue())
            return self.next_track()
        return self.next_track()
```

```
    def show(self):
        """Строка вида «A | ►B | C» - текущий трек отмечен значком ►."""
        parts, node = [], self.tracks.head
        while node is not None:
            parts.append(("►" if node is self.current else "") + str(node.value))
            node = node.next
        return " | ".join(parts) if parts else "(пусто)"


# ---------------------------------------------------------------------------
# 5. САМОПРОВЕРКА И МАЛЫЙ ПРИМЕР (T1)
# ---------------------------------------------------------------------------
def self_test():
    """
    Сверяет двусвязный список со встроенным list, а кольцевую очередь - с
    collections.deque на случайных операциях. При ошибке - AssertionError.
    """
    rng = random.Random(0)
    for _ in range(300):                            # много коротких случайных сценариев
        dll, ref = DoublyLinkedList(), []
        for _ in range(60):
            op = rng.choice(["pf", "pb", "ins", "popf", "popb", "del", "get"])
            v = rng.randint(0, 99)
            if op == "pf":
                dll.push_front(v)
                ref.insert(0, v)
            elif op == "pb":
                dll.push_back(v)
                ref.append(v)
            elif op == "ins":
                i = rng.randint(0, len(ref))
                dll.insert(i, v)
                ref.insert(i, v)
            elif op == "popf" and ref:
                assert dll.pop_front() == ref.pop(0)
            elif op == "popb" and ref:
                assert dll.pop_back() == ref.pop()
            elif op == "del" and ref:
                i = rng.randrange(len(ref))
                assert dll.delete(i) == ref.pop(i)
            elif op == "get" and ref:
                i = rng.randrange(len(ref))
                assert dll.get(i) == ref[i] == dll.get_from_head(i)
            # после каждой операции проверяем содержимое в обе стороны и длину
            assert dll.to_list() == ref
            assert dll.to_list_reversed() == ref[::-1]
            assert len(dll) == len(ref)
    for cap in (1, 2, 5, 16):                       # кольцевая очередь против deque
        q, ref = CircularQueue(cap), deque()
        for _ in range(500):
            if rng.random() < 0.5 and len(ref) < cap:
                v = rng.randint(0, 99)
                q.enqueue(v)
                ref.append(v)
            elif ref:
                assert q.dequeue() == ref.popleft()
            assert q.to_list() == list(ref) and len(q) == len(ref)
    print("Самопроверка пройдена (список и очередь совпадают с list и deque)")


def demo_small():
    """T1: малый пример - содержимое структур выводится после КАЖДОЙ операции."""
    print("\n--- T1. Малый пример: двусвязный список ---")
    dll = DoublyLinkedList()

    def show(action):
        print(f"{action:<28} вперёд: {dll.to_list()}  назад: {dll.to_list_reversed()}  длина: {len(dll)}")

    for v in (10, 20, 30, 40, 50, 60, 70, 80):
        dll.push_back(v)
    show("push_back 10..80")
    dll.push_front(5)
    show("push_front(5)")
    dll.insert(len(dll) // 2, 99)
    show(f"insert({(len(dll) - 1) // 2}, 99) - середина")
    dll.insert(1, 11)
    show("insert(1, 11)")
    mid = dll._node_at(len(dll) // 2)
    dll.insert_after(mid, 77)
    show(f"insert_after(узла {mid.value}, 77)")
    print(f"{'get(4) / find(77)':<28} {dll.get(4)} / {dll.find(77)}")
    dll.pop_front()
    show("pop_front()")
    dll.pop_back()
    show("pop_back()")
```

```
    dll.delete(3)
    show("delete(3)")

    print("\n--- T1. Малый пример: кольцевая очередь (ёмкость 5) ---")
    q = CircularQueue(5)

    def showq(action):
        print(f"{action:<22} очередь: {q.to_list()}  head={q.head}  count={q.count}  буфер: {q.data}")

    for t in ("заявка-1", "заявка-2", "заявка-3", "заявка-4"):
        q.enqueue(t)
        showq(f"enqueue({t})")
    print(f"{'dequeue()':<22} -> {q.dequeue()}")
    showq("после dequeue")
    print(f"{'dequeue()':<22} -> {q.dequeue()}")
    showq("после dequeue")
    for t in ("заявка-5", "заявка-6", "заявка-7"):   # tail переходит через конец массива
        q.enqueue(t)
        showq(f"enqueue({t})")
    try:
        q.enqueue("заявка-8")
    except OverflowError as e:
        print(f"{'enqueue(заявка-8)':<22} -> {e}")


def demo_playlist():
    """Прикладная задача: сценарий работы с плей-листом (вывод после каждого шага)."""
    print("\n--- Прикладная задача: плей-лист с переходами ---")
    pl = Playlist(["Трек A", "Трек B", "Трек C", "Трек D", "Трек E"])

    def step(action, result=None):
        suffix = "" if result is None else f"  -> играет: {result}"
        print(f"{action:<34} {pl.show()}{suffix}")

    step("начальное состояние")
    step("next_track()", pl.next_track())
    step("next_track()", pl.next_track())
    step("prev_track()", pl.prev_track())
    pl.add_after_current("Трек X")
    step("add_after_current('Трек X')")
    pl.queue_next("Трек Y")
    pl.queue_next("Трек Z")
    print(f"{'queue_next('+chr(39)+'Трек Y'+chr(39)+'), Z':<34} очередь: {pl.up_next.to_list()}")
    step("play_next() (из очереди)", pl.play_next())
    step("play_next() (из очереди)", pl.play_next())
    step("play_next()", pl.play_next())
    pl.remove_current()
    step("remove_current()", pl.now_playing())
    for _ in range(3):
        pl.next_track()
    step("3 x next_track() - конец", pl.now_playing())
    step("next_track() - к началу (цикл)", pl.next_track())
    step("prev_track() - к концу (цикл)", pl.prev_track())


# ---------------------------------------------------------------------------
# 6. ИЗМЕРЕНИЯ (T2)
# ---------------------------------------------------------------------------
def median_time(run, reps=REPS):
    """
    Медиана по reps повторам. run() сама измеряет нужный участок и
    возвращает время в секундах (подготовка данных в замер не входит).
    """
    run()                                           # «прогревочный» запуск: его результат отбрасывается
    return statistics.median(run() for _ in range(reps))


def timed(func):
    """
    Время выполнения func() в секундах. На время замера выключается сборщик
    мусора, чтобы его случайные срабатывания не искажали измерения.
    """
    gc.collect()                                    # заранее убираем накопившийся мусор
    gc.disable()
    try:
        t0 = time.perf_counter()
        func()
        return time.perf_counter() - t0
    finally:
        gc.enable()


def build_dll(n):
    """Двусвязный список из чисел 0..n-1."""
    dll = DoublyLinkedList()
```

```
    for i in range(n):
        dll.push_back(i)
    return dll


def measure_mid_insert(n):
    """Вставка в середину: время ОДНОЙ вставки (DLL по индексу, DLL после узла, list)."""
    def run_dll():
        dll = build_dll(n)

        def work():
            for _ in range(K_MID):
                dll.insert(len(dll) // 2, 0)
        return timed(work)

    def run_node():
        dll = build_dll(n)
        node = dll._node_at(n // 2)                # ссылка на средний узел уже есть

        def work():
            for _ in range(K_NODE):
                dll.insert_after(node, 0)
        return timed(work)

    def run_list():
        lst = list(range(n))

        def work():
            for _ in range(K_MID * 10):            # у list операция быстрая - берём больше повторов
                lst.insert(len(lst) // 2, 0)
        return timed(work)

    dll = build_dll(n)                             # отдельный прогон для счётчика шагов
    for _ in range(K_MID):
        dll.insert(len(dll) // 2, 0)
    return {
        "dll_index": median_time(run_dll) / K_MID,
        "dll_node": median_time(run_node) / K_NODE,
        "list": median_time(run_list) / (K_MID * 10),
        "hops_per_op": dll.hops / K_MID,
    }


def measure_traversal(n):
    """Обход всех n элементов: вперёд, назад, «назад через индексы от головы» и list."""
    dll = build_dll(n)
    lst = list(range(n))

    def run_fwd():
        return timed(lambda: [None for _ in dll])

    def run_back():
        return timed(lambda: [None for _ in dll.reversed()])

    def run_naive():                               # так выглядел бы обход назад без ссылок prev
        def work():
            for i in range(n - 1, -1, -1):
                dll.get_from_head(i)
        return timed(work)

    def run_list_fwd():
        return timed(lambda: [None for _ in lst])

    def run_list_back():
        return timed(lambda: [None for _ in reversed(lst)])

    dll.hops = 0
    for i in range(n - 1, -1, -1):
        dll.get_from_head(i)
    naive_hops = dll.hops                          # фиксируем сразу: сумма 0 + 1 + ... + (n - 1)
    return {
        "fwd": median_time(run_fwd),
        "back": median_time(run_back),
        "naive_back": median_time(run_naive, reps=3),
        "list_fwd": median_time(run_list_fwd),
        "list_back": median_time(run_list_back),
        "naive_hops": naive_hops,
    }


def measure_access(n):
    """Доступ по индексу: время одного обращения к случайной позиции (DLL и list)."""
    rng = random.Random(SEED + n)
    idx = [rng.randrange(n) for _ in range(K_ACCESS)]
    dll = build_dll(n)
```

```
    lst = list(range(n))
    dll.hops = 0
    for i in idx:
        dll.get(i)
    hops = dll.hops / K_ACCESS
    return {
        "dll": median_time(lambda: timed(lambda: [dll.get(i) for i in idx])) / K_ACCESS,
        "list": median_time(lambda: timed(lambda: [lst[i] for i in idx])) / K_ACCESS,
        "hops_per_op": hops,
    }


def measure_queue(n):
    """Очередь: n постановок и n извлечений (суммарное время), кольцо / list.pop(0) / deque."""
    def run_ring():
        q = CircularQueue(n)

        def work():
            for i in range(n):
                q.enqueue(i)
            for _ in range(n):
                q.dequeue()
        return timed(work)

    def run_list():
        lst = []

        def work():
            for i in range(n):
                lst.append(i)
            for _ in range(n):
                lst.pop(0)                          # извлечение из начала - сдвиг, Θ(n)
        return timed(work)

    def run_deque():
        d = deque()

        def work():
            for i in range(n):
                d.append(i)
            for _ in range(n):
                d.popleft()
        return timed(work)

    return {"ring": median_time(run_ring, 3), "list_pop0": median_time(run_list, 3),
            "deque": median_time(run_deque, 3)}


def measure_list_mid_ext(n):
    """Время одной вставки в середину list для больших n (подтверждение Θ(n))."""
    lst = list(range(n))
    reps = 200

    def work():
        for _ in range(reps):
            lst.insert(len(lst) // 2, 0)
    return statistics.median(timed(work) for _ in range(3)) / reps


def slope(sizes, times):
    """Показатель k в T ~ n^k: наклон линейной регрессии log T от log n."""
    xs = [math.log(s) for s in sizes]
    ys = [math.log(t) for t in times]
    mx, my = statistics.mean(xs), statistics.mean(ys)
    return sum((x - mx) * (y - my) for x, y in zip(xs, ys)) / sum((x - mx) ** 2 for x in xs)


# ---------------------------------------------------------------------------
# 7. ПАМЯТЬ (Т4)
# ---------------------------------------------------------------------------
def deep_size_list(lst):
    """Размер list вместе с объектами int внутри (sys.getsizeof не «глубокий»)."""
    return sys.getsizeof(lst) + sum(sys.getsizeof(x) for x in lst)


def deep_size_dll(dll):
    """Размер двусвязного списка: все узлы Node плюс объекты int."""
    total, node = sys.getsizeof(dll), dll.head
    while node is not None:
        total += sys.getsizeof(node) + sys.getsizeof(node.value)
        node = node.next
    return total


def deep_size_ring(q):
```

```
    """Размер кольцевой очереди: массив-буфер плюс объекты int."""
    return sys.getsizeof(q) + sys.getsizeof(q.data) + sum(
        sys.getsizeof(x) for x in q.data if x is not None)


def peak_memory(builder):
    """Память (байты), занятая объектом, который строит builder(): по tracemalloc."""
    tracemalloc.start()
    obj = builder()
    current, _ = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    del obj
    return current


def experiment_memory():
    """Т4: расход памяти на MEM_N элементов для list, двусвязного списка и очереди."""
    n = MEM_N
    values = list(range(n))
    lst = list(values)
    dll = build_dll(n)
    q = CircularQueue(n)
    for v in values:
        q.enqueue(v)
    ints = sum(sys.getsizeof(x) for x in values)

    def build_q():
        qq = CircularQueue(n)
        for v in range(n):
            qq.enqueue(v)
        return qq

    return {
        "n": n,
        "int_size": sys.getsizeof(12345),
        "node_size": sys.getsizeof(dll.head),
        "list_getsizeof": deep_size_list(lst),
        "dll_getsizeof": deep_size_dll(dll),
        "ring_getsizeof": deep_size_ring(q),
        "ints_total": ints,
        "list_tracemalloc": peak_memory(lambda: list(range(n))),
        "dll_tracemalloc": peak_memory(lambda: build_dll(n)),
        "ring_tracemalloc": peak_memory(build_q),
    }


# ---------------------------------------------------------------------------
# 8. ГРАФИКИ
# ---------------------------------------------------------------------------
def make_plots(res):
    """Строит и сохраняет рисунки lr6_plot.png и lr6_plot_memory.png."""
    sizes = SIZES
    fig, axes = plt.subplots(2, 2, figsize=(11, 8))

    ax = axes[0][0]                                  # вставка в середину
    mi = res["mid_insert"]
    ax.plot(sizes, [mi[str(n)]["dll_index"] * 1e6 for n in sizes], "o-", label="DLL: insert(i, v)")
    ax.plot(sizes, [mi[str(n)]["dll_node"] * 1e6 for n in sizes], "s-", label="DLL: insert_after(узел)")
    ax.plot(sizes, [mi[str(n)]["list"] * 1e6 for n in sizes], "^-", label="list.insert(i, v)")
    ax.set_title("Вставка в середину (одна операция)")
    ax.set_ylabel("мкс")

    ax = axes[0][1]                                  # обход
    tr = res["traversal"]
    ax.plot(sizes, [tr[str(n)]["fwd"] * 1e3 for n in sizes], "o-", label="DLL вперёд (next)")
    ax.plot(sizes, [tr[str(n)]["back"] * 1e3 for n in sizes], "s-", label="DLL назад (prev)")
    ax.plot(sizes, [tr[str(n)]["naive_back"] * 1e3 for n in sizes], "d-", label="назад по индексам от головы")
    ax.plot(sizes, [tr[str(n)]["list_fwd"] * 1e3 for n in sizes], "^-", label="list вперёд")
    ax.set_title("Обход всех n элементов")
    ax.set_ylabel("мс")

    ax = axes[1][0]                                  # доступ по индексу
    ac = res["access"]
    ax.plot(sizes, [ac[str(n)]["dll"] * 1e6 for n in sizes], "o-", label="DLL: get(i)")
    ax.plot(sizes, [ac[str(n)]["list"] * 1e6 for n in sizes], "^-", label="list[i]")
    ax.set_title("Доступ по индексу (одно обращение)")
    ax.set_ylabel("мкс")

    ax = axes[1][1]                                  # очередь
    qs = sorted(int(k) for k in res["queue"])
    ax.plot(qs, [res["queue"][str(n)]["ring"] * 1e3 for n in qs], "o-", label="кольцевая очередь")
    ax.plot(qs, [res["queue"][str(n)]["deque"] * 1e3 for n in qs], "s-", label="collections.deque")
    ax.plot(qs, [res["queue"][str(n)]["list_pop0"] * 1e3 for n in qs], "^-", label="list.pop(0)")
    ax.set_xscale("log")
    ax.set_yscale("log")
```

```
    ax.set_title("Очередь: n постановок + n извлечений")
    ax.set_ylabel("мс (лог. шкала)")

    for ax in axes.flat:
        ax.set_xlabel("n")
        ax.grid(True, alpha=0.3)
        ax.legend(fontsize=8)
    fig.tight_layout()
    fig.savefig("lr6_plot.png", dpi=150)
    plt.close(fig)

    # --- память ------------------------------------------------------------
    m = res["memory"]
    fig, ax = plt.subplots(figsize=(6.5, 4))
    labels = ["list", "Двусвязный\nсписок", "Кольцевая\nочередь"]
    vals = [m["list_getsizeof"], m["dll_getsizeof"], m["ring_getsizeof"]]
    bars = ax.bar(labels, [v / 1024 for v in vals], color=["tab:blue", "tab:orange", "tab:green"])
    for b, v in zip(bars, vals):
        ax.text(b.get_x() + b.get_width() / 2, b.get_height(), f"{v / m['n']:.0f} Б/эл.",
                ha="center", va="bottom", fontsize=9)
    ax.set_ylabel("КиБ (sys.getsizeof, глубокий подсчёт)")
    ax.set_title(f"Память на {m['n']:,} элементов".replace(",", " "))
    fig.tight_layout()
    fig.savefig("lr6_plot_memory.png", dpi=150)
    plt.close(fig)


# ---------------------------------------------------------------------------
# 9. ВЫВОД РЕЗУЛЬТАТОВ
# ---------------------------------------------------------------------------
def system_info():
    """Строка с характеристиками системы (вставляется в отчёт)."""
    cpu = platform.processor() or platform.machine()
    try:                                          # на Linux модель процессора лежит в /proc/cpuinfo
        with open("/proc/cpuinfo", encoding="utf-8") as f:
            for line in f:
                if line.startswith("model name"):
                    cpu = line.split(":", 1)[1].strip()
                    break
    except OSError:
        pass
    return (f"ОС: {platform.system()} {platform.release()}; процессор: {cpu}; "
            f"Python {platform.python_version()}")


def print_report(res):
    """Печатает результаты измерений в консоль."""
    mi, tr, ac, qu = res["mid_insert"], res["traversal"], res["access"], res["queue"]
    print("\nТаблица 1. Вставка в середину: время одной операции, мкс (и число переходов по ссылкам)")
    print(f"  {'n':>6}{'DLL insert(i)':>16}{'переходов':>11}{'DLL после узла':>16}{'list.insert':>14}")
    for n in SIZES:
        r = mi[str(n)]
        print(f"  {n:>6}{r['dll_index'] * 1e6:>16.2f}{r['hops_per_op']:>11.0f}"
              f"{r['dll_node'] * 1e6:>16.3f}{r['list'] * 1e6:>14.3f}")
    print("  list.insert в середину для больших n (мкс):",
          ", ".join(f"n={n}: {t * 1e6:.1f}" for n, t in res["list_mid_ext"].items()))

    print("\nТаблица 2. Обход всех n элементов, мс")
    print(f"  {'n':>6}{'DLL вперёд':>12}{'DLL назад':>12}{'назад от головы':>17}{'переходов':>12}"
          f"{'list вперёд':>13}{'list назад':>12}")
    for n in SIZES:
        r = tr[str(n)]
        print(f"  {n:>6}{r['fwd'] * 1e3:>12.3f}{r['back'] * 1e3:>12.3f}{r['naive_back'] * 1e3:>17.2f}"
              f"{r['naive_hops']:>12}{r['list_fwd'] * 1e3:>13.3f}{r['list_back'] * 1e3:>12.3f}")

    print("\nТаблица 3. Доступ по индексу: время одного обращения, мкс (и переходов)")
    print(f"  {'n':>6}{'DLL get(i)':>12}{'переходов':>11}{'list[i]':>10}")
    for n in SIZES:
        r = ac[str(n)]
        print(f"  {n:>6}{r['dll'] * 1e6:>12.3f}{r['hops_per_op']:>11.1f}{r['list'] * 1e6:>10.4f}")

    print("\nТаблица 4. Очередь: n постановок + n извлечений, мс")
    print(f"  {'n':>7}{'кольцевая':>12}{'deque':>10}{'list.pop(0)':>14}")
    for n in sorted(int(k) for k in qu):
        r = qu[str(n)]
        print(f"  {n:>7}{r['ring'] * 1e3:>12.2f}{r['deque'] * 1e3:>10.2f}{r['list_pop0'] * 1e3:>14.2f}")

    print("\nТаблица 5. T3: отношение T(2n)/T(n) и показатель k в T ~ n^k")
    for name, ser in res["scaling"].items():
        print(f"  {name:<44} T(2n)/T(n): {', '.join(f'{x:.2f}' for x in ser['ratios'])};  k = {ser['k']:.2f}")

    m = res["memory"]
    print(f"\nТаблица 6. T4: память на {m['n']} элементов (int = {m['int_size']} Б, узел Node = {m['node_size']} Б)")
    for key, name in (("list", "list"), ("dll", "двусвязный список"), ("ring", "кольцевая очередь")):
        g, t = m[f"{key}_getsizeof"], m[f"{key}_tracemalloc"]
```

```
        print(f"  {name:<20} getsizeof: {g:>9} Б ({g / m['n']:.1f} Б/эл.);  tracemalloc: {t:>9} Б ({t / m['n']:.1f} Б/эл.)")


def make_scaling(res):
    """Т3: для каждой серии измерений считает T(2n)/T(n) и показатель наклона k."""
    mi, tr, ac = res["mid_insert"], res["traversal"], res["access"]
    series = {
        "DLL: вставка в середину по индексу": [mi[str(n)]["dll_index"] for n in SIZES],
        "DLL: вставка после известного узла": [mi[str(n)]["dll_node"] for n in SIZES],
        "list.insert в середину": [mi[str(n)]["list"] for n in SIZES],
        "DLL: обход вперёд": [tr[str(n)]["fwd"] for n in SIZES],
        "DLL: обход назад (prev)": [tr[str(n)]["back"] for n in SIZES],
        "Обход назад по индексам от головы": [tr[str(n)]["naive_back"] for n in SIZES],
        "list: обход вперёд": [tr[str(n)]["list_fwd"] for n in SIZES],
        "DLL: доступ по индексу": [ac[str(n)]["dll"] for n in SIZES],
        "list[i]": [ac[str(n)]["list"] for n in SIZES],
        "Кольцевая очередь (n + n операций)": [res["queue"][str(n)]["ring"] for n in SIZES],
    }
    out = {}
    for name, times in series.items():
        out[name] = {"ratios": [times[i + 1] / times[i] for i in range(len(times) - 1)],
                     "k": slope(SIZES, times)}
    return out


def main():
    print("Лабораторная работа № 6, вариант 13")
    print(system_info())
    print(f"Структуры: двусвязный список, кольцевая очередь; n = {SIZES}; повторов: {REPS} (медиана)")
    self_test()
    demo_small()
    demo_playlist()

    res = {}
    print("\nИзмерения (T2) ...", flush=True)
    res["mid_insert"] = {str(n): measure_mid_insert(n) for n in SIZES}
    res["list_mid_ext"] = {str(n): measure_list_mid_ext(n) for n in EXT_LIST_SIZES}
    res["traversal"] = {str(n): measure_traversal(n) for n in SIZES}
    res["access"] = {str(n): measure_access(n) for n in SIZES}
    res["queue"] = {str(n): measure_queue(n) for n in SIZES + EXT_QUEUE_SIZES}
    res["scaling"] = make_scaling(res)
    res["memory"] = experiment_memory()
    print_report(res)
    make_plots(res)
    print("\nГрафики сохранены: lr6_plot.png, lr6_plot_memory.png")
    if "--json" in sys.argv:                        # служебный вывод для отчёта
        path = sys.argv[sys.argv.index("--json") + 1]
        with open(path, "w", encoding="utf-8") as f:
            json.dump(res, f, ensure_ascii=False, indent=1)
    if "--show" in sys.argv:
        plt.show()


if __name__ == "__main__":
    main()
```

5. Характеристики вычислительной системы

Параметр	Значение
Процессор	Intel(R) Xeon(R) Processor @ 2.10GHz, 2 логических ядра
Оперативная память	8 ГБ
Операционная система	Linux (Ubuntu 24.04)
Интерпретатор	Python 3.13.18
Таймер	time.perf_counter()

Абсолютные значения времени зависят от системы; число переходов по ссылкам, показатели роста и соотношения между структурами должны сохраниться. Программа при запуске выводит характеристики системы автоматически.

6. Результаты

6.1. Проверка на малом примере (T1)

Содержимое структур выводится после каждой операции (для списка — в обе стороны, для очереди — также состояние буфера). Результаты соответствуют ожидаемым.

```
--- T1. Малый пример: двусвязный список ---
push_back 10..80             вперёд: [10, 20, 30, 40, 50, 60, 70, 80]  назад: [80, 70, 60, 50, 40, 30, 20, 10]  длина: 8
push_front(5)                вперёд: [5, 10, 20, 30, 40, 50, 60, 70, 80]  назад: [80, 70, 60, 50, 40, 30, 20, 10, 5]  длина: 9
insert(4, 99) - середина     вперёд: [5, 10, 20, 30, 99, 40, 50, 60, 70, 80]  назад: [80, 70, 60, 50, 40, 99, 30, 20, 10, 5]  длина: 10
insert(1, 11)                вперёд: [5, 11, 10, 20, 30, 99, 40, 50, 60, 70, 80]  назад: [80, 70, 60, 50, 40, 99, 30, 20, 10, 11, 5]
      длина: 11
insert_after(узла 99, 77)    вперёд: [5, 11, 10, 20, 30, 99, 77, 40, 50, 60, 70, 80]  назад: [80, 70, 60, 50, 40, 77, 99, 30, 20, 10,
      11, 5]  длина: 12
get(4) / find(77)            30 / 6
pop_front()                  вперёд: [11, 10, 20, 30, 99, 77, 40, 50, 60, 70, 80]  назад: [80, 70, 60, 50, 40, 77, 99, 30, 20, 10, 11]
      длина: 11
pop_back()                   вперёд: [11, 10, 20, 30, 99, 77, 40, 50, 60, 70]  назад: [70, 60, 50, 40, 77, 99, 30, 20, 10, 11]  длина:
      10
delete(3)                    вперёд: [11, 10, 20, 99, 77, 40, 50, 60, 70]  назад: [70, 60, 50, 40, 77, 99, 20, 10, 11]  длина: 9

--- T1. Малый пример: кольцевая очередь (ёмкость 5) ---
enqueue(заявка-1)      очередь: ['заявка-1']  head=0  count=1  буфер: ['заявка-1', None, None, None, None]
enqueue(заявка-2)      очередь: ['заявка-1', 'заявка-2']  head=0  count=2  буфер: ['заявка-1', 'заявка-2', None, None, None]
enqueue(заявка-3)      очередь: ['заявка-1', 'заявка-2', 'заявка-3']  head=0  count=3  буфер: ['заявка-1', 'заявка-2', 'заявка-3',
      None, None]
enqueue(заявка-4)      очередь: ['заявка-1', 'заявка-2', 'заявка-3', 'заявка-4']  head=0  count=4  буфер: ['заявка-1', 'заявка-2',
      'заявка-3', 'заявка-4', None]
dequeue()              -> заявка-1
после dequeue          очередь: ['заявка-2', 'заявка-3', 'заявка-4']  head=1  count=3  буфер: [None, 'заявка-2', 'заявка-3',
      'заявка-4', None]
dequeue()              -> заявка-2
после dequeue          очередь: ['заявка-3', 'заявка-4']  head=2  count=2  буфер: [None, None, 'заявка-3', 'заявка-4', None]
enqueue(заявка-5)      очередь: ['заявка-3', 'заявка-4', 'заявка-5']  head=2  count=3  буфер: [None, None, 'заявка-3', 'заявка-4',
      'заявка-5']
enqueue(заявка-6)      очередь: ['заявка-3', 'заявка-4', 'заявка-5', 'заявка-6']  head=2  count=4  буфер: ['заявка-6', None,
      'заявка-3', 'заявка-4', 'заявка-5']
enqueue(заявка-7)      очередь: ['заявка-3', 'заявка-4', 'заявка-5', 'заявка-6', 'заявка-7']  head=2  count=5  буфер: ['заявка-6',
      'заявка-7', 'заявка-3', 'заявка-4', 'заявка-5']
enqueue(заявка-8)      -> Очередь переполнена
```

6.2. Прикладная задача: плей-лист (сценарий)

Знаком ► отмечен текущий трек. Видно: переходы вперёд и назад, вставка после текущего, очередь «играть следующим», удаление текущего и зацикливание на границах.

```
--- Прикладная задача: плей-лист с переходами ---
начальное состояние              ►Трек A | Трек B | Трек C | Трек D | Трек E
next_track()                     Трек A | ►Трек B | Трек C | Трек D | Трек E  -> играет: Трек B
next_track()                     Трек A | Трек B | ►Трек C | Трек D | Трек E  -> играет: Трек C
prev_track()                     Трек A | ►Трек B | Трек C | Трек D | Трек E  -> играет: Трек B
add_after_current('Трек X')      Трек A | ►Трек B | Трек X | Трек C | Трек D | Трек E
queue_next('Трек Y'), Z          очередь: ['Трек Y', 'Трек Z']
play_next() (из очереди)         Трек A | Трек B | ►Трек Y | Трек X | Трек C | Трек D | Трек E  -> играет: Трек Y
play_next() (из очереди)         Трек A | Трек B | Трек Y | ►Трек Z | Трек X | Трек C | Трек D | Трек E  -> играет:
      Трек Z
play_next()                      Трек A | Трек B | Трек Y | Трек Z | ►Трек X | Трек C | Трек D | Трек E  -> играет:
      Трек X
remove_current()                 Трек A | Трек B | Трек Y | Трек Z | ►Трек C | Трек D | Трек E  -> играет: Трек C
3 x next_track() - конец         ►Трек A | Трек B | Трек Y | Трек Z | Трек C | Трек D | Трек E  -> играет: Трек A
next_track() - к началу (цикл)   Трек A | ►Трек B | Трек Y | Трек Z | Трек C | Трек D | Трек E  -> играет: Трек B
prev_track() - к концу (цикл)    ►Трек A | Трек B | Трек Y | Трек Z | Трек C | Трек D | Трек E  -> играет: Трек A
```

6.3. Измерения времени операций (T2)

Таблица 2. Вставка в середину: время одной операции, мкс

n	Двусвязный список: insert(i, v)	Переходов по ссылкам на операцию	Двусвязный список: insert_after(узел)	list.insert(i, v)
1 000	8,76	524	0,17	0,22
2 000	17,56	1024	0,34	0,27
4 000	35,39	2024	0,18	0,46

Для проверки роста `list.insert` в середину (время одной операции, мкс) на больших n: n = 10 000: 0,9; n = 100 000: 10,7; n = 1 000 000: 159,9.

Таблица 3. Обход всех n элементов, мс

n	DLL вперёд (next)	DLL назад (prev)	Назад «по индексам от головы»	Переходов в этом случае	list вперёд	list назад
1 000	0,049	0,048	7,99	499 500	0,024	0,025
2 000	0,084	0,080	34,56	1 999 000	0,040	0,040
4 000	0,152	0,148	134,54	7 998 000	0,064	0,070

Таблица 4. Доступ по индексу: время одного обращения, мкс

n	DLL: get(i)	Переходов в среднем	list[i]
1 000	3,77	249,8	0,070
2 000	7,85	500,9	0,064
4 000	17,68	1019,5	0,074

Таблица 5. Очередь: n постановок и n извлечений (суммарное время), мс

n	Кольцевая очередь	collections.deque	list.append + list.pop(0)
1 000	0,31	0,08	0,09
2 000	0,59	0,13	0,23
4 000	1,21	0,26	0,68
16 000	4,71	0,91	20,94
64 000	18,98	3,42	361,52

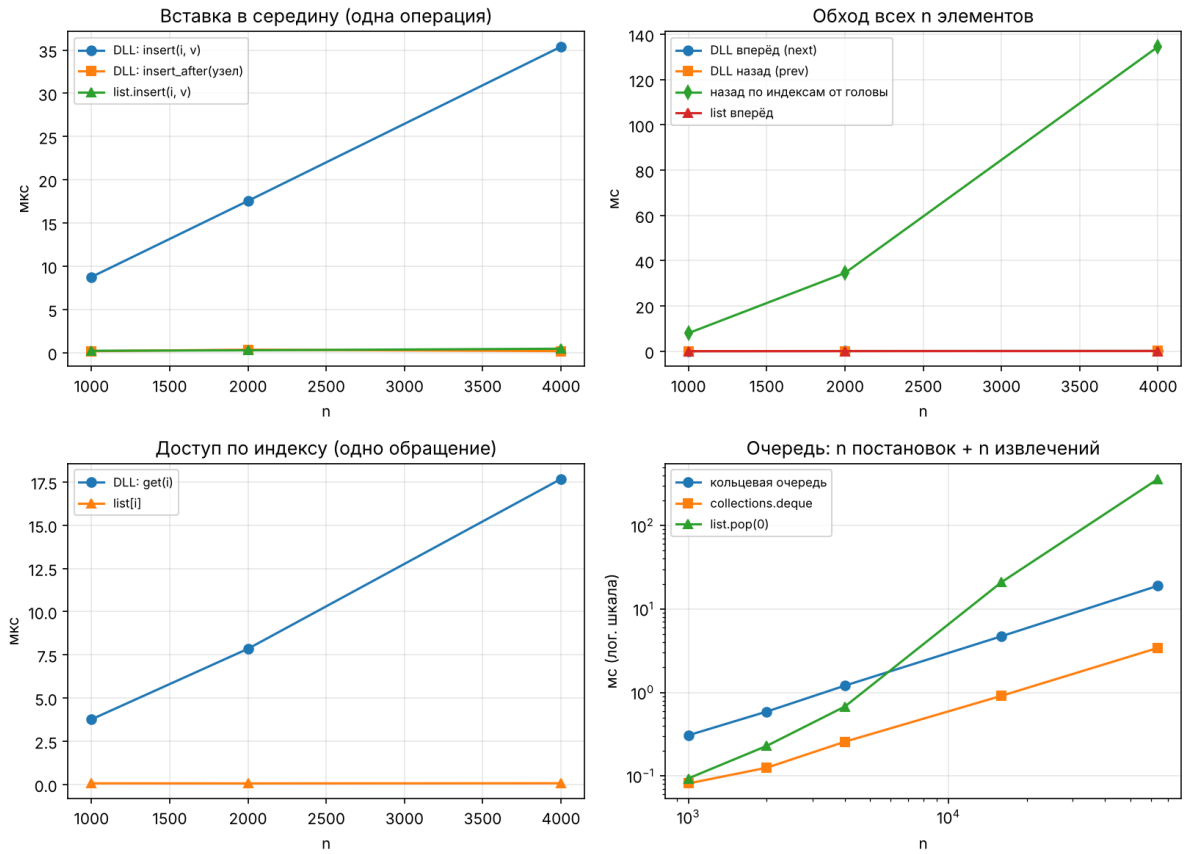


Рис. 8. Графики T(n): вставка в середину, обход, доступ по индексу, очередь (для очереди — логарифмические шкалы)

6.4. Расход памяти (T4)

Таблица 6. Память на 10 000 элементов (int = 28 Б, узел Node = 56 Б)

Структура	sys.getsizeof (глубокий подсчёт), Б	Б на элемент	tracemalloc, Б	Б на элемент
list	360 056	36,0	391 832	39,2
Двусвязный список	840 048	84,0	871 912	87,2
Кольцевая очередь (ёмкость 10 000)	360 104	36,0	391 912	39,2

sys.getsizeof не учитывает вложенные объекты, поэтому размеры контейнера, всех узлов и всех объектов int суммировались вручную; значение tracemalloc приведено для проверки.

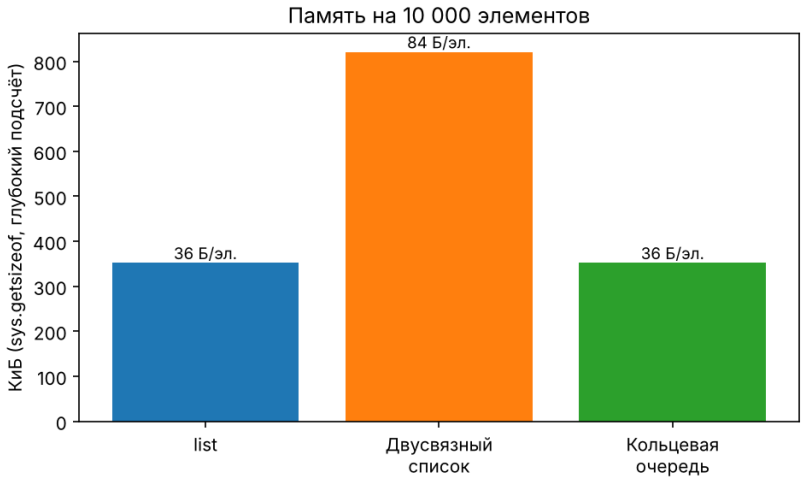


Рис. 9. Расход памяти на 10 000 элементов

7. Сводная таблица «операция — структура — теоретическая оценка — эксперимент»

T(2n)/T(n) — отношения времени при удвоении n (n = 1000→2000 и 2000→4000); k — показатель в $T \sim n^k$ по трём точкам. Для Θ(1) ожидается отношение ≈ 1, для Θ(n) — ≈ 2, для $\Theta(n^2)$ — ≈ 4.

Операция	Структура	Теория	T(2n)/T(n)	k	Соответствие
Вставка в середину по индексу	Двусвязный список	Θ(n)	2,01; 2,02	1,01	Да; число переходов 524 / 1024 / 2024 ≈ n/2
Вставка после известного узла	Двусвязный список	Θ(1)	2,00; 0,51	0,02	Да: роста нет (0 переходов), время ≈ 0,2–0,35 мкс — на уровне шума
Вставка в середину	list	Θ(n)	1,25; 1,67	0,53	Да, но при n ≤ 4000 доминирует фиксированная стоимость вызова; на n = 10^4…10^6 рост линейный
Обход вперёд	Двусвязный список	Θ(n)	1,70; 1,82	0,81	Да (≈ линейно; малое n — фиксированные издержки)
Обход назад	Двусвязный список (prev)	Θ(n)	1,66; 1,86	0,82	Да (≈ линейно)
Обход назад без ссылок prev	Доступ по индексу от головы	$\Theta(n^2)$	4,33; 3,89	2,04	Да: ≈ 4, переходов n(n − 1)/2
Обход вперёд	list	Θ(n)	1,68; 1,59	0,71	Да
Доступ по индексу	Двусвязный список	Θ(n)	2,08; 2,25	1,11	Да; в среднем 250 / 501 / 1020 переходов ≈ n/4
Доступ по индексу	list	Θ(1)	0,92; 1,16	0,05	Да: время не зависит от n
n постановок + n извлечений	Кольцевая очередь	Θ(n) всего, Θ(1) на операцию	1,92; 2,05	0,99	Да; при 16 000 → 64 000 отношение 4,0 (ожидалось 4)
n постановок + n извлечений	list: append + pop(0)	$\Theta(n^2)$ всего	—	—	Да: при 16 000 → 64 000 время выросло в 17,3 раз (ожидалось 16)

Для очень быстрых операций (доли микросекунды) и малых n измерение содержит фиксированную составляющую (вызов функции, создание списка-аккумулятора), поэтому показатели k у них смещены; надёжным подтверждением служат число переходов по ссылкам и измерения на больших n.

8. Обоснование выбора структуры для задачи «плей-лист с переходами»

Основные действия пользователя: переход к следующему и предыдущему треку, вставка трека сразу после текущего («играть следующим»), удаление текущего трека, зацикливание на границах плей-листа. Все они выполняются **относительно текущей позиции**, то есть узел известен заранее.

Требование	Двусвязный список	Односвязный список	Массив (list)
Переход вперёд	Θ(1) по next	Θ(1)	Θ(1) (индекс + 1)
Переход назад	**Θ(1) по prev**	Θ(n) — проход с головы	Θ(1)
Вставка после текущего	**Θ(1)**: 4 ссылки	Θ(1)	Θ(n): сдвиг хвоста
Удаление текущего	**Θ(1)**	Θ(n): нужен предыдущий узел	Θ(n): сдвиг
Доступ к треку по номеру	Θ(n) (до n/2)	Θ(n)	Θ(1)

Выбор. Двусвязный список — единственная из рассмотренных структур, у которой *все* действия вокруг текущего трека стоят Θ(1): переход назад обеспечивает ссылка prev (в односвязном списке он стоил бы Θ(n), а полный обход назад — $\Theta(n^2)$, что подтверждено измерением: 135 мс против 0,15 мс при n = 4000), а вставка и удаление сводятся к перестановке ссылок без сдвига. Зацикливание реализуется переходом от хвоста к голове (и наоборот) за Θ(1). Очередь «играть следующим» естественно строится на кольцевом буфере: постановка и извлечение за Θ(1), ёмкость ограничена (в плеере это несколько ближайших треков).

Оговорка. Двусвязный список проигрывает при доступе к треку по номеру (Θ(n): 1020 переходов в среднем при n = 4000, 17,68 мкс против 0,07 мкс у list) и расходует больше памяти (2,3 раза). Для небольших плей-листов (сотни-тысячи треков) выигрыш на практике незаметен: встроенный list.insert в середину на n = 4000 занимает 0,46 мкс благодаря быстрому копированию памяти в C. Преимущество списка проявляется при частом редактировании больших списков, при хранении ссылки на текущий узел и там, где важна гарантированная трудоёмкость Θ(1) независимо от размера. Если бы требовался частый «прыжок» на трек по номеру, лучше был бы массив.

9. Выводы

1. **Теория подтверждена.** Вставка в середину двусвязного списка по индексу — Θ(n): время растёт при удвоении n в 2,01 и 2,02 раза (8,76 → 17,56 → 35,39 мкс), а число переходов по ссылкам равно 524, 1024, 2024 (≈ n/2 — поиск позиции с ближайшего конца). Вставка после *известного* узла от n не зависит (Θ(1)): 0 переходов, ≈ 0,2–0,35 мкс — примерно на два порядка быстрее вставки по индексу при n = 4000.
2. **Главное преимущество связного списка — не вставка «по номеру», а вставка и удаление в известной позиции.** Если позиция задана индексом, список не лучше массива: дойти до неё стоит Θ(n). Встроенный `list.insert` тоже линеен (при n = 10^4, 10^5, 10^6: 0,9, 10,7, 160 мкс), но на малых n он быстрее списка из-за маленькой константы (сдвиг памяти в C): при n = 4000 0,46 мкс против 35,39

мкс у списка.

3. **Обход в обе стороны — Θ(n) и симметричен.** Обход вперёд по `next` и назад по `prev` занимают одинаковое время (при n = 4000: 0,152 и 0,148 мс), рост при удвоении n ≈ 1,70–1,82. Без ссылки prev обход назад через доступ по индексу от головы стоит $\Theta(n^2)$: 7 998 000 переходов и 135 мс при n = 4000 (в 906 раз медленнее), отношение времён при удвоении n равно 4,33 и 3,89 (теория 4). Это главное оправдание второй ссылки.
4. **Доступ по индексу** у двусвязного списка — Θ(n) (в среднем 250, 501, 1020 переходов ≈ n/4 благодаря выбору ближайшего конца), у массива — Θ(1) (0,07 мкс при любом n против 17,68 мкс у списка при n = 4000). Если требуется частый доступ по номеру, выбирается массив.
5. **Кольцевая очередь** выполняет постановку и извлечение за Θ(1): суммарное время n + n операций растёт линейно (отношения 1,92 и 2,05), память не выделяется заново, элементы не сдвигаются. Очередь на `list` с `pop(0)` квадратична: при n = 64 000 она в 19 раз медленнее кольцевой (361,52 против 18,98 мс), хотя при n = 1000–4000 разница мала или обратная. Реализация на чистом Python при этом уступает C-реализации `collections.deque` в 5,6 раза при n = 64 000 — учебная реализация нужна для понимания устройства, в практике используется deque.
6. **Память (Т4).** На 10 000 элементов list занимает 360 056 Б (36 Б на элемент: 8 Б ссылка + 28 Б объект int), двусвязный список — 840 048 Б (84 Б на элемент), то есть в 2,3 раза больше: каждый узел — отдельный объект (56 Б: заголовок объекта и три ссылки — значение, prev, next) вместо одной ячейки в массиве. Кольцевая очередь расходует столько же, сколько list (36 Б на элемент), так как это массив фиксированной ёмкости. Данные tracemalloc отличаются от getsizeof менее чем на 10 %. Кроме того, узлы размещены в памяти произвольно, что ухудшает использование кэша процессора: обход списка медленнее обхода массива в 2,4 раза при n = 4000 несмотря на одинаковую асимптотику.
7. **Плей-лист.** Для задачи с переходами вперёд и назад, вставкой после текущего трека и удалением текущего оптимален двусвязный список (все операции Θ(1) относительно текущего узла) в сочетании с кольцевой очередью для списка «играть следующим». Правильность реализации проверена самопроверкой на случайных операциях в сравнении с `list` и `deque` и сценарием, приведённым в разделе 6.2.

10. Список литературы

1. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2013. — Гл. 10.
2. Кнут Д. Э. Искусство программирования. Т. 1. Основные алгоритмы. — 3-е изд. — М.: Вильямс, 2018. — Гл. 2.2.
3. Седжвик Р., Уэйн К. Алгоритмы на Java. — 4-е изд. — М.: Вильямс, 2013. — Гл. 1.3.
4. Документация Python: collections.deque; time complexity [Электронный ресурс]. — URL: https://wiki.python.org/moin/TimeComplexity